

Anatomia pratica del motore

Dai «pezzi di testo» del modulo 1 a come sono fatti davvero: token, vocabolario, embedding — e il fatto che il modello non ricorda nulla da una chiamata all'altra.

Cosa sono le «parole» per un modello

- Il motore non lavora su caratteri e nemmeno su parole: lavora su **token**, unità di testo prese da un elenco fisso.
- Un token può essere una parola intera, un pezzo di parola, un segno di punteggiatura o uno spazio iniziale.
- Internamente non è nemmeno testo: è un numero intero, l'indice di quel token nell'elenco.
- La previsione del modulo 1 avviene quindi su interi: «quale indice viene dopo?».

LO STESSO TESTO, TRE LIVELLI

testo "Ordine spedito"

token ["Ord", "ine", " sped", "ito"]

id [8974, 483, 26195, 2118]

Il vocabolario del modello

- È una tabella fissa token $\leftrightarrow$ id , decisa prima dell'addestramento e mai più modificata.
- Dimensione tipica: da ~32.000 a ~130.000 voci. Ogni previsione è una scelta fra tutte quelle voci.
- Il vocabolario riflette i testi con cui è stato costruito: l'inglese entra in pochi token, l'italiano tecnico in molti di più.
- Parole assenti non danno errore: vengono ricomposte da frammenti presenti.

ESTRATTO DEL VOCABOLARIO

1043	" the"
8974	"0rd"
26195	" sped"
220	" " (spazio)
128001	< end_of_text >

Anche i marcatori di controllo (fine testo, inizio turno) sono voci del vocabolario.

Tokenizzazione: spezzare il testo in token

MAT-4471/B → 7 TOKEN

MAT - 44 71 / B + 1 marcatore di inizio

- Regola pratica: l'algoritmo unisce le sequenze di caratteri più frequenti nei dati e lascia isolato il resto.
- Perché il modello sbaglia a contare le lettere: non le vede affatto, vede blocchi come 44 e 71.
- È deterministica e sta fuori dal modello: stesso testo, sempre gli stessi token.
- Impatto sui vostri dati: codici, matricole e ID SAP si frammentano, e frammenti simili diventano confondibili.

Dai token ai numeri: gli embedding

- L'id da solo non dice niente: 8974 non è «più grande» di 483. Serve una rappresentazione con dentro del significato.
- Ogni id punta a una riga di una tabella: un vettore di centinaia o migliaia di numeri in virgola mobile.
- Quella tabella non è scritta a mano: i valori si formano durante l'addestramento, insieme al resto dei pesi.
- Da qui in poi il motore fa solo algebra lineare su questi vettori.

LA RIGA DI LOOKUP

```
emb[8974] = [ 0.0121, -0.3384, 0.9042,  
-0.0077, 0.4510, ... ]
```

Lunghezza tipica del vettore: 768 – 4096 numeri. Una dimensione singola non ha un significato leggibile.

Lo spazio semantico: vicinanza = somiglianza di significato

- I vettori non sono sparsi a caso: token usati in contesti simili finiscono vicini.
- «Vicino» è una misura calcolabile: la similarità cosinusoidale fra due vettori, un numero fra -1 e 1.
- È somiglianza d'uso, non un sinonimo da dizionario: «fattura» e «bolla» sono vicine perché appaiono negli stessi discorsi.
- Questa proprietà è ciò che nel modulo 3 permetterà di cercare per significato invece che per parola esatta.

`COS_SIM(V1, V2)`

"server" · "macchina"		0.78
"server" · "rete"		0.61
"server" · "lasagna"		0.09

Vedere uno spazio a centinaia di dimensioni: la riduzione dimensionale (PCA)

- Un vettore da 4096 numeri non è disegnabile: per guardarlo lo si proietta su 2 o 3 assi.
- La PCA sceglie gli assi lungo cui i dati variano di più: si conserva la struttura d'insieme, non i dettagli.
- È una proiezione con perdita: due punti vicini sulla mappa possono essere lontani nello spazio originale.
- Vale come strumento di lettura per noi, non come qualcosa che il modello usa.

DALLA MATRICE ALLA MAPPA

X (1200 × 4096) `PCA(2)` X_{2d} (1200 × 2)

Le due colonne finali non sono «animalità» e «tecnicità»: sono combinazioni delle 4096 originali.

Cluster che emergono dai dati, non programmati a mano

- Nessuno ha scritto una tassonomia: i gruppi tematici si formano perché quelle parole compaiono negli stessi contesti.
- Conseguenza: i confini sono sfumati e dipendono dai dati di addestramento, non da una regola aziendale.
- Termini interni e sigle proprietarie cadono dove capita: il modello non conosce il vostro dominio.

GRUPPI TIPICI IN UNA MAPPA 2D

animali

tecnologia

cibo

emozioni

Le parole nuove che aggiungeremo in demo si posizionano da sole, senza che nessuno indichi il gruppo.

Il modello è stateless: nessuna memoria tra una chiamata e l'altra

COME SEMBRA

«Si ricorda che prima gli ho detto il numero d'ordine.»

COM'È

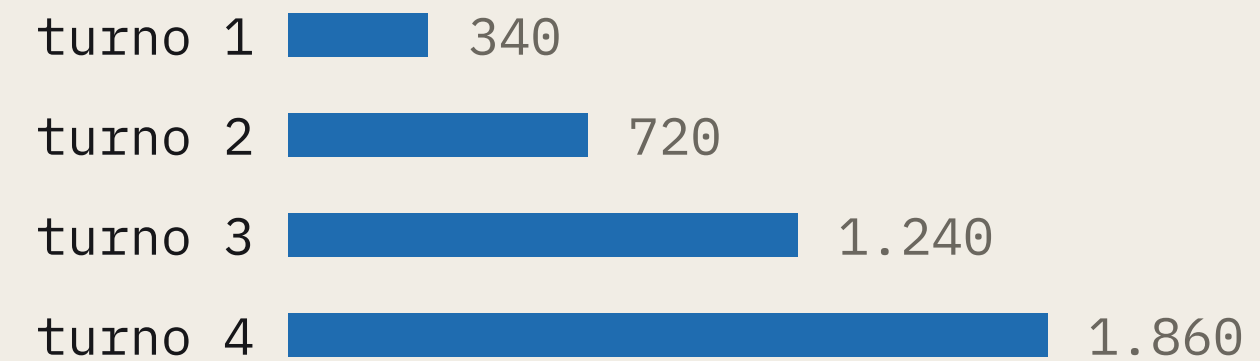
L'applicazione gli ha rispedito l'intera conversazione, numero d'ordine incluso.

- Ogni chiamata è indipendente: il motore è una funzione senza sessione, senza cache di conversazione, senza identità dell'utente.
- La memoria è una responsabilità applicativa: la tiene l'orchestratore visto nel modulo 1, non il modello.
- Implicazione di security: ciò che rientra nel prompt lo decide il vostro codice, ed è lì che si fa il controllo.

La context window: reinviare tutto ad ogni turno

- La context window è il testo che il modello ha davanti in questa chiamata: tutto ciò che non è lì, per lui non esiste.
- Ogni turno rispedisce l'intera cronologia più la nuova domanda: la finestra cresce, non si aggiorna in modo incrementale.
- Costo e latenza salgono con la lunghezza della conversazione, non con la difficoltà della domanda.

TOKEN INVIATI PER TURNO



Il system prompt: istruzioni permanenti nascoste in ogni chiamata

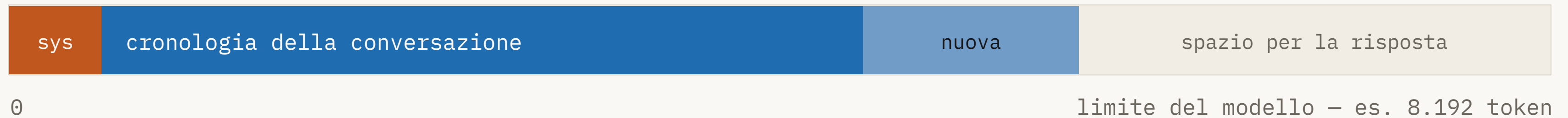
- È un blocco di testo che l'applicazione mette in cima alla finestra prima dei messaggi dell'utente, a ogni chiamata.
- Contiene ruolo, tono, vincoli e strumenti disponibili: è lì che vive la «personalità» di un assistente aziendale.
- Occupa token come qualsiasi altro testo: un system prompt lungo consuma finestra e budget a ogni turno.
- Non è un livello di sicurezza: sono istruzioni, e come istruzioni possono essere aggirate dal testo dell'utente.

IN CIMA A OGNI RICHIESTA

```
{ "role": "system ", "content": "Sei  
l'assistente interno. Rispondi solo su  
procedure di produzione. Non citare fonti  
esterne." }
```

~40 token, rispediti identici a ogni turno.

Il limite fisico della context window (token massimi)



- Il limite è una proprietà dell'architettura, non un'impostazione: input e output competono per lo stesso spazio.
- È da qui che nasce l'«amnesia» a metà di una chat lunga: quel pezzo di testo non è più nella finestra.
- Superato il tetto non arriva un errore utile: l'applicazione tronca o riassume la cronologia più vecchia.
- Corollario: infilare un manuale intero nel prompt non scala. Serve scegliere cosa mandare — modulo 3.

La vera chiamata al modello: il payload JSON reale

```
POST /v1/chat/completions {"model":  
  "llama-3-8b-instruct", "messages": [  
    {"role": "system", "content": "..."},  
    {"role": "user", "content": "..."},  
    {"role": "assistant", "content": "..."},  
    {"role": "user", "content": "..."} ], "temperature": 0.2  
  }
```

- Nessuna sessione, nessun id di conversazione: una normale POST senza stato, ripetuta a ogni turno.
- `messages` è la context window in forma serializzata: la memoria è questo array, costruito dal vostro codice.
- La temperatura del modulo 1 è un campo qui, alla pari del testo.
- Tutto ciò che finisce in questo JSON esce dalla vostra rete verso il motore: è il punto in cui si applicano le policy sui dati.

Verso il modulo 3: conoscenza congelata al training, dati aziendali mai visti

COSA SA

Quello che era nei testi di addestramento, fino a una certa data. Poi i pesi si fermano.

COSA NON SA

Il vostro MES, gli ordini di ieri, le procedure interne: non erano nei dati e non ci entreranno.

- L'unico canale verso i dati aziendali è la context window: se non ce li mettiamo noi, il modello li inventa.
- Ma la finestra è piccola: va scelto il pezzo di documentazione giusto, non spedito tutto.
- Modulo 3 — RAG: cercare per significato con gli embedding di oggi e iniettare solo i frammenti utili.